About the New Architecture

Since 2018, the React Native team has been redesigning the core internals of React Native to enable developers to create higher-quality experiences. As of 2024, this version of React Native has been proven at scale and powers production apps by Meta.

The term *New Architecture* refers to both the new framework architecture and the work to bring it to open source.

The New Architecture has been available for experimental opt-in as of React Native 0.68 with continued improvements in every subsequent release. The team is now working to make this the default experience for the React Native open source ecosystem.

Why a New Architecture?

After many years of building with React Native, the team identified a set of limitations that prevented developers from crafting certain experiences with a high polish. These limitations were fundamental to the existing design of the framework, so the New Architecture started as an investment in the future of React Native.

The New Architecture unlocks capabilities and improvements that were impossible in the legacy architecture.

Synchronous Layout and Effects

Building adaptive UI experiences often requires measuring the size and position of your views and adjusting layout.

Today, you would use the `onLayout` event to get the layout information of a view and make any adjustments. However, state updates within the `onLayout` callback may apply after painting the previous render. This means that users may see intermediate states or visual jumps between rendering the initial layout and responding to layout measurements.

With the New Architecture, we can avoid this issue entirely with synchronous access to layout information and properly scheduled updates such that no intermediate state is visible to users.

- Example: Rendering a Tooltip

Support for Concurrent Renderer and Features

The New Architecture supports concurrent rendering and features that have shipped in React 18 and beyond. You can now use features like Suspense for data-fetching, Transitions, and other new React APIs in your React Native code, further conforming codebases and concepts between web and native React development.

The concurrent renderer also brings out-of-the-box improvements like automatic batching, which reduces re-renders in React.

- Example: Automatic Batching

New concurrent features, like Transitions, give you the power to express the priority of UI updates. Marking an update as lower priority tells React it can "interrupt" rendering the update to handle higher priority updates to ensure a responsive user experience where it matters.

- Example: Using `startTransition`

Fast JavaScript/Native Interfacing

The New Architecture removes the asynchronous bridge between JavaScript and native and replaces it with JavaScript Interface (JSI). JSI is an interface that allows JavaScript to

hold a reference to a C++ object and vice-versa. With a memory reference, you can directly invoke methods without serialization costs.

JSI enables VisionCamera, a popular camera library for React Native, to process frames in real time. Typical frame buffers are ~30 MB, which amounts to roughly 2 GB of data per second, depending on the frame rate. In comparison with the serialization costs of the bridge, JSI handles that amount of interfacing data with ease. JSI can expose other complex instance-based types such as databases, images, audio samples, etc.

JSI adoption in the New Architecture removes this class of serialization work from all native-JavaScript interop. This includes initializing and re-rendering native core components like `View` and `Text`. You can read more about our investigation in rendering performance in the New Architecture and the improved benchmarks we measured.

What can I expect from enabling the New Architecture?

While the New Architecture enables these features and improvements, enabling the New Architecture for your app or library may not immediately improve the performance or user experience.

For example, your code may need refactoring to leverage new capabilities like synchronous layout effects or concurrent features. Although JSI will minimize the overhead between JavaScript and native memory, data serialization may not have been a bottleneck for your app's performance.

Enabling the New Architecture in your app or library is opting into the future of React Native.

The team is actively researching and developing new capabilities the New Architecture unlocks. For example, web alignment is an active area of exploration at Meta that will ship to the React Native open source ecosystem.

- Updates to the event loop model
- Node and layout APIs

- Styling and layout conformance

You can follow along and contribute in our dedicated discussions & proposals repository.

Should I use the New Architecture today?

With 0.76, The New Architecture is enabled by default in all the React Native projects.

If you find anything that is not working well, please open an issue using this template.

If, for any reasons, you can't use the New Architecture, you can still opt-out from it:

Android

1. Open the `android/gradle.properties` file
2. Toggle the `newArchEnabled` flag from `true` to `false`

gradle.properties

```
# Use this property to enable support to the new architecture.
# This will allow you to use TurboModules and the Fabric render in
# your application. You should enable this flag either if you want
# to write custom TurboModules/Fabric components OR use libraries that
# are providing them.
-newArchEnabled=true
+newArchEnabled=false
```

iOS

1. Open the `ios/Podfile` file
2. Add `ENV['RCT_NEW_ARCH_ENABLED'] = '0'` in the main scope of the Podfile (reference Podfile in the template)

diff

```
```

```
+ ENV['RCT_NEW_ARCH_ENABLED'] = '0'
# Resolve react_native_pods.rb with node to allow for hoisting
require Pod::Executable.execute_command('node', ['-p',
  'require.resolve(
```

3. Install your CocoaPods dependencies with the command:

```
bundle exec pod install
```

Is this page useful? 

Edit this page

*Last updated on **Mar 22, 2026***